

Dinámica y movimiento para videojuegos y gamificación

Tema 1. Conceptos Básicos

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Unidades de medida
- 3 Representación gráfica en 3D
- 4 Vectores
- 5 Matrices
- 6 Derivación numérica
- 7 Integración numérica
- 8 Resolución numérica de ecuaciones diferenciales

1 Introducción

La Física en los videojuegos

La Física sostiene la credibilidad e inmersión de los mundos virtuales y convierte una simple animación en una experiencia convincente.

- **Evita que “parezcan falsos”.** Los jugadores esperan comportamientos coherentes con la realidad: pilotar, lanzar una pelota o mover un personaje debe parecer correcto correcto.
- **El realismo es más fácil de lo que parece.** Predominan ecuaciones algebraicas y trigonométricas; los casos complejos (fluidos, aerodinámica) usan EDOs resueltas numéricamente.
- **No penaliza el rendimiento.** El coste dominante suele ser el *rendering*, no la física; los modelos razonables se ejecutan sin problemas en hardware actual.
- **Mejora tus habilidades como desarrollador.** Entender los fundamentos físicos guía qué efectos incluir y cómo implementarlos con eficiencia (ej.: efecto de una bola de golf, fricción, gravedad, colisiones).

¿Para qué necesito saber Física si ya existen motores físicos?

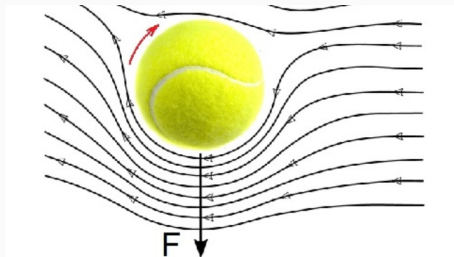
Los motores físicos (*Unity Physics*, *Unreal Chaos*, *Box2D*, *Bullet*, *Chipmunk2D* etc.) resuelven automáticamente colisiones, fuerzas, integración, contactos, fricción, etc. Pero **no toman decisiones de diseño**: solo ejecutan lo que tú les pides.

¿Por qué necesitas saber Física entonces?

- **Para ajustar el comportamiento**: masa, amortiguamiento, fricción, rebote... Sin entender su fundamento físico, es imposible afinarlos correctamente.
- **Para detectar y resolver problemas**: jitter, penetración, explosiones numéricas, cuerpos que salen disparados, acumulación de energía, errores de integración...
- **Para saber qué motor usar y cómo configurarlo**: raycasts, rigid bodies, constraints, joints, integradores, capas de colisión, etc.

¿Para qué necesito saber Física si ya existen motores físicos?

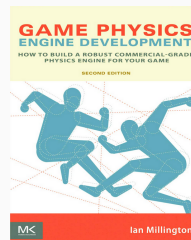
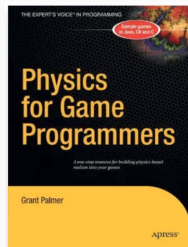
- **Para crear efectos que el motor no trae “de fábrica”:** viento, fuerzas aerodinámicas, balística realista, partículas físicas, ragdolls creíbles...
- **Para no depender ciegamente del motor:** los juegos buenos requieren **control**, no milagros.



Referencias recomendadas

Libros de Física

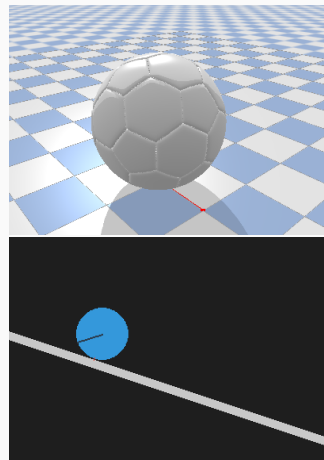
- Palmer, Grant. *Physics for Game Programmers*. Apress 1st. ed., 2005
- Bourg, David M. *Physics for Game Developers*. O'Reilly Media, 2002.
- Millington, Ian. *Game Physics Engine Development*. CRC Press, 2010.



Referencias recomendadas

Physics engines

- **Bullet Physics SDK** Erwin Coumans. GitHub repository.
<https://github.com/bulletphysics/bullet3>
- **Chipmunk** Scott Lembcke. *Chipmunk2D: A fast and lightweight 2D game physics library*.
Version 7.0.3, MIT License, 2019.
<https://github.com/slembcke/Chipmunk2D>
- Aquí usaremos wrappers para Python (al menos yo)
 - Pybullet para Bullet
<https://pypi.org/project/pybullet/>
 - Pymunk para Chipmunk
<https://pypi.org/project/pymunk/>



2 Unidades de medida

Unidades de medida

- Para describir el mundo físico, lo que hacemos es medir las distintas magnitudes de interés (masa, velocidad, etc).
- Para medir cada tipo de magnitud, necesitamos una **unidad**.
- Normalmente, se toman como magnitudes fundamentales la longitud $[L]$, la masa $[M]$ y el tiempo $[T]$. El resto de magnitudes se derivan a partir de estas. A la dependencia de una magnitud derivada con respecto a las fundamentales se le llama las **dimensiones** de la unidad derivada. **Ejemplo:** la velocidad tiene las dimensiones $[L]/[T]$.
- Al conjunto formado por las unidades usadas para medir las distintas magnitudes se le llama **Sistema de unidades**. Nosotros usaremos el **Sistema Internacional (S.I.)**.
- El S.I. también se llama **MKS**, porque las unidades de las magnitudes fundamentales son el metro, el kilogramo y el segundo.

Sistema Internacional

Magnitud	Dim.	S.I.
Longitud	$[L]$	m
Tiempo	$[T]$	s
Masa	$[M]$	kg
Velocidad	$[L][T]^{-1}$	m/s
Aceleración	$[L][T]^{-2}$	m/s ²
Fuerza	$[M][L][T]^{-2}$	N
Presión	$[M][L]^{-1}[T]^{-2}$	Pa

Magnitud	Dim.	S.I.
Energía	$[M][L]^2[T]^{-2}$	J
Potencia	$[M][L]^2[T]^{-3}$	W
Superficie	$[L]^2$	m ²
Volumen	$[L]^3$	m ³
Densidad	$[M][L]^{-3}$	kg/m ³
Cant. movimiento	$[M][L][T]^{-1}$	kg·m/s
Impulso	$[M][L][T]^{-1}$	N·s
Frecuencia	$[T]^{-1}$	Hz

3 Representación gráfica en 3D

Sistemas de coordenadas

- En juegos y simulaciones, gran parte de la física se reduce a **posicionar** y **mover** objetos en el espacio.
- Usaremos **coordenadas cartesianas** por defecto y **esféricas** cuando convenga (cámaras/orbitas).
- Trabajaremos con **escalares** y **vectores**; veremos sus **operaciones** básicas y dos productos clave: **producto escalar** y **producto vectorial**.

Sistemas de coordenadas

Cartesianas (x, y, z)

- Convención típica en juegos: ejes x e y paralelos al suelo; z hacia arriba.
- Fáciles de visualizar y de integrar con motores de render.

Esféricas (r, θ, φ)

- Útiles en órbitas y cámaras “alrededor” de un objetivo.
- r : **Distancia al origen** (radial)
- θ : **Ángulo cenital** (desde $+Z$), $\theta \in [0, \pi)$
- φ : **Azimut en el plano XY** desde $+x$, $\varphi \in [0, 2\pi]$.

Ejemplos de coordenadas esféricas en videojuegos

- Cámara en tercera persona orbitando al personaje.
- Posición de un enemigo en “radar” (distancia y ángulos).
- proyectiles con trayectoria circular/helicoidal.

Conversión entre coordenadas esféricas y cartesianas

De esféricas a cartesianas

$$x = r \sin \theta \cos \varphi$$

$$y = r \sin \theta \sin \varphi$$

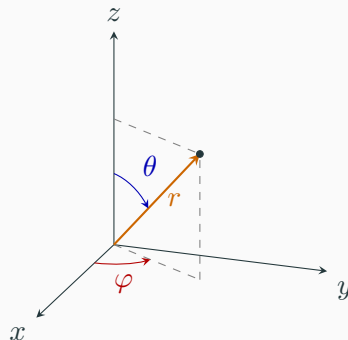
$$z = r \cos \theta$$

De cartesianas a esféricas

Distancia radial $r = \sqrt{x^2 + y^2 + z^2}$

Ángulo cenital $\theta = \arccos\left(\frac{z}{r}\right)$

Ángulo acimutal $\varphi = \text{atan2}(y, x)$



Convención: θ desde eje z (polar)

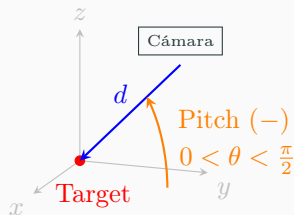
La cámara en PyBullet

- En la práctica, los motores físicos como **PyBullet** usan una variante de las coordenadas esféricas para controlar la cámara.
- `cameraDistance` (d): Igual que r . Es la distancia de la cámara al punto de observación.
- `cameraYaw` (ψ): Similar a φ pero $\psi = 0$ cuando la cámara está sobre el eje Y negativo y mira hacia el eje Y positivo:
 - $\psi = \varphi + \pi/2$ por lo que también crece en sentido antihorario
 - $\psi \in (-\pi, \pi]$
- `cameraPitch` (α): Similar a θ pero se mide desde el plano XY
 - $\alpha = \theta - \pi/2$. Es negativo por encima del plano y positivo por debajo.
 - $\alpha \in [-\pi/2, \pi/2]$

La cámara en PyBullet: Pitch

Interpretación de la Cámara

- Estas coordenadas corresponden a la posición relativa de la cámara con respecto al objeto. Si la cámara está arriba, ve al objeto abajo.
- **Pitch (-) = Mirar abajo:** Cuando la cámara está arriba, $\theta \in [0, \pi/2]$, el Pitch es negativo
 $\alpha = \theta - \pi/2 \in [-\pi/2, 0]$. Al aumentar el Pitch, la cámara sube en altura sobre el horizonte. Para mantener el *Target* centrado, la lente debe inclinarse hacia abajo.
- **Distancia (d):** Es el radio de la esfera de orbitación.



El sistema no pregunta **¿Dónde está el objeto?**,
 sino **¿Desde dónde estoy grabando al objeto?**

La cámara en PyBullet. Conversión entre coordenadas.

IMPORTANTE recordar que hablamos de las coordenadas de la posición de la cámara, relativa al objeto.

De PyBullet a cartesianas

$$x = d \cos \alpha \sin \psi$$

$$y = -d \cos \alpha \cos \psi$$

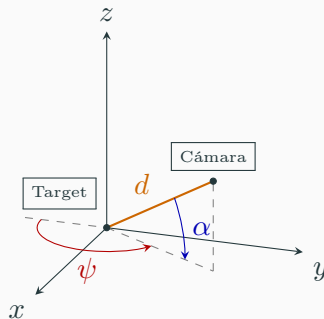
$$z = -d \sin \alpha$$

De cartesianas a PyBullet

Distance $d = \sqrt{x^2 + y^2 + z^2}$

Pitch $\alpha = -\arcsin\left(\frac{z}{d}\right)$

Yaw $\psi = \text{atan2}(x, -y)$



Convención PyBullet: α (Pitch) desde plano XY y ψ (Yaw) desde eje $-Y$.

La cámara en PyBullet. Unidades

¡Ojo con las unidades de medida!

Contexto	Ángulos	Funciones
Cálculo físico de vectores, velocidades angulares, etc.	Radianes	<code>np.sin()</code> , <code>np.cos()</code> , etc.
Visualización PyBullet	Grados	<code>resetDebugVisualizerCamera()</code>

- Para convertir: $\text{grados} = \text{radianes} \times 180/\pi$

Implementación en PyBullet

```
1  import pybullet as p
2  import pybullet_data
3  import time
4  # Configuración inicial
5  p.connect(p.GUI)
6  p.setAdditionalSearchPath(pybullet_data.getDataPath())
7  p.setGravity(0, 0, -9.81)
8  # Cargar suelo y cubo pequeño
9  p.loadURDF("plane.urdf")
10 cubo_id = p.loadURDF("cube_small.urdf", [0, 0, 0.03])
11 # Ajustar cámara: yaw=180 mira hacia Y- desde Y+
12 p.resetDebugVisualizerCamera(cameraDistance=0.173, cameraYaw=180, \
13     cameraPitch=-45, cameraTargetPosition=[0, 0, 0])
14 # Bucle de simulación
15 while True:
16     p.stepSimulation()
17     time.sleep(1./240.)
18
```

4 Vectores

Magnitudes escalares y vectoriales

Escalares

- Quedan definidos por un único número (masa, tiempo, temperatura).
- Operan con aritmética usual.

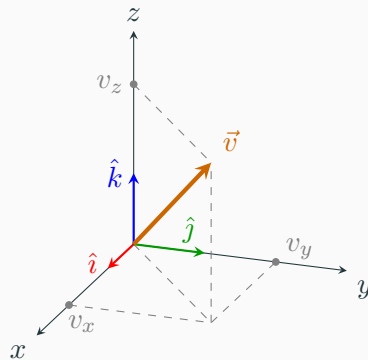
Vectores

- Tienen **módulo**, **dirección** y **sentido** (velocidad, fuerza, aceleración).
- En $\mathbb{R}^3$: $\vec{v} = (v_x, v_y, v_z)$.
- **Módulo y vector unitario**

$$\|\vec{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}, \quad \hat{v} = \frac{\vec{v}}{\|\vec{v}\|} \quad (\|\hat{v}\| = 1)$$

Descomposición en ejes

$$\vec{v} = v_x \hat{i} + v_y \hat{j} + v_z \hat{k}$$



Operaciones con vectores (básicas)

Suma: $\vec{a} + \vec{b} = (a_x + b_x, a_y + b_y, a_z + b_z)$

Escalar: $\lambda \vec{a} = (\lambda a_x, \lambda a_y, \lambda a_z)$

Distancia: $d(\vec{p}, \vec{q}) = \|\vec{p} - \vec{q}\|$

Proyección de $\vec{a}$ sobre $\vec{b}$: $\text{proj}_{\vec{b}}(\vec{a}) = \left(\frac{\vec{a} \cdot \vec{b}}{\|\vec{b}\|^2} \right) \vec{b}$

Usos en simulación

- Suma/escala: acumulación de fuerzas, interpolaciones (*lerp*).
- Proyección: alinear movimiento con una superficie.

En el ejemplo anterior de PyBullet le pasamos a `resetDebugVisualizerCamera` la posición del objeto como un vector de tres componentes: `cameraTargetPosition=[0, 0, 0]`.

```

1      p.resetDebugVisualizerCamera(
2          cameraDistance=0.173,
3          cameraYaw=180,
4          cameraPitch=-45,
5          cameraTargetPosition=[0,0,0]
6      )
7  
```

Producto escalar

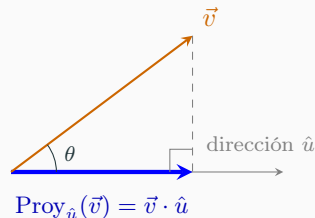
$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z = \|\vec{a}\| \|\vec{b}\| \cos \theta$$

Propiedades

- Conmuta y es lineal: $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$, $\vec{a} \cdot (\lambda \vec{b}) = \lambda(\vec{a} \cdot \vec{b})$.
- Ortogonalidad: $\vec{a} \cdot \vec{b} = 0 \iff \theta = 90^\circ$.

Aplicaciones en simulación

- *Ángulo entre vectores*: comprobación de campo de visión (FOV) o “aim assist”.
- *Proyección de velocidad*: velocidad tangencial = $\vec{v} \cdot \hat{t}$.
- *Potencia*: $P = \vec{F} \cdot \vec{v}$ (útil para depurar fuerzas y movimientos).



Producto vectorial

$$\vec{a} \times \vec{b} = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

Módulo y dirección

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \|\vec{b}\| \sin \theta \quad (\text{área del paralelogramo}).$$

Dirección dada por la **regla de la mano derecha** (ortogonal a ambos).

Aplicaciones en simulación

- Cálculo de **normales** (iluminación, físicas de contacto).
- **Torque** y momentos angulares en *rigid bodies* 3D.



Mini-ejemplos prácticos

1) Cámara orbital sobre objetivo $\vec{p}_t = (x_t, y_t, z_t)$:

La posición de la cámara $\vec{p}_c$ se obtiene sumando a la posición del objetivo el vector de desplazamiento relativo en esféricas.

$$x_c = x_t + r \sin \theta \cos \varphi$$

$$y_c = y_t + r \sin \theta \sin \varphi$$

$$z_c = z_t + r \cos \theta$$

2) Alineación de movimiento (proyectar $\vec{v}$ sobre “forward” $\hat{f}$):

Descomposición de la velocidad en componentes paralela y perpendicular a la dirección de avance.

$$\vec{v}_{\parallel} = (\vec{v} \cdot \hat{f}) \hat{f}$$

$$\vec{v}_{\perp} = \vec{v} - \vec{v}_{\parallel}$$

3) Normal de una superficie (vía tangentes $\vec{t}_1, \vec{t}_2$):

Cálculo del vector unitario perpendicular al plano definido por dos direcciones tangenciales.

$$\hat{n} = \frac{\vec{t}_1 \times \vec{t}_2}{\|\vec{t}_1 \times \vec{t}_2\|}$$

5 Matrices

Introducción a las Matrices

- Un **matriz** es un objeto matemático utilizado para almacenar conjuntos de datos de manera estructurada.
- Aunque se podrían usar variables separadas, las matrices permiten expresiones matemáticas y código de programas más **compactos y eficientes**. Por ejemplo un sistema de N ecuaciones lineales con N incógnitas se escribe como una sola ecuación matricial.
- En el desarrollo de videojuegos, se utilizan comúnmente para:
 - Modelar el vuelo de aviones.
 - Simular colisiones entre objetos.
 - Resolver sistemas de ecuaciones con múltiples incógnitas.

Sistemas de Ecuaciones en Forma Matricial

Consideremos un sistema de tres ecuaciones con tres incógnitas (x, y, z) :

En forma matricial se expresa como:

$$a = 4x + y - 3z$$

$$b = -7x - 2y + 5z$$

$$c = 3x + 3y + z$$

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} = A \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 4 & 1 & -3 \\ -7 & -2 & 5 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- Esta es una matriz de **dos dimensiones** con tres filas y tres columnas.
- Para resolver el sistema de ecuaciones “basta” con obtener la inversa de la matriz A : A^{-1}
- Las matrices pueden ser de cualquier dimensión (1D, 2D, 3D,...).

Operaciones con Matrices

Suma y Resta

Implica sumar o restar los elementos individuales. Las matrices **deben tener las mismas dimensiones**.

Multiplicación

- Se multiplica cada fila de la primera matriz por cada columna de la segunda.
- **Regla fundamental:** El número de columnas de A debe ser igual al número de filas de B .

Sean las matrices:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix}$$

El elemento c_{11} del producto $AB = C$ se calcula como:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31}$$

Matrices de Rotación

Se utilizan para rotar temporalmente los ejes de coordenadas y realizar cálculos, como en colisiones de palos de golf y pelotas.

Si rotamos los ejes x e y un ángulo α sobre el **eje z** :

$$\begin{pmatrix} v'_x \\ v'_y \\ v'_z \end{pmatrix} = \begin{pmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$$

- El ángulo θ se conoce también como **ángulo de Euler**.
- Como la rotación es sobre el eje z , la componente v'_z permanece inalterada ($v'_z = v_z$).

Ángulos de Euler

Definición

Representan la orientación de un cuerpo rígido mediante tres rotaciones sucesivas (α, β, γ) respecto a ejes locales o globales.

1. **Precesión** (α): Rotación sobre el eje Z .
2. **Nutación** (β): Rotación sobre el nuevo eje X (eje de nodos).
3. **Rotación propia** (γ): Rotación final sobre el eje Z del cuerpo.

Matriz R

$$R = R_z \cdot R_{x'} \cdot R_{z''}$$

- **Grados de libertad:** Cubren los 3 grados de rotación en $\mathbb{R}^3$.

Matrices de Rotación de Euler

La matriz de rotación total es el producto $R = R_z(\alpha)R_x(\beta)R_z(\gamma)$:

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$R_x(\beta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{pmatrix}$$

$$R_z(\gamma) = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Nota Importante

El orden de multiplicación es crucial. En rotaciones **intrínsecas** (sobre ejes móviles), las matrices se multiplican de izquierda a derecha.

Cálculo de la Trayectoria Final

Ejemplo: Un avión vuela en la dirección del eje Y positivo y vira 20° a estribor y se eleva 30° . Obténgase el nuevo rumbo del avión.

El vector de rumbo inicial será: $\vec{v}_0 = (0, 1, 0)^T$. Aplicamos dos rotaciones sucesivas definidas por los ángulos α y β :

- **Rotación α (sobre eje z):** Giro horizontal ($\alpha = -20^\circ$).
- **Rotación β (sobre eje x'):** Elevación vertical ($\beta = 30^\circ$).

$$\vec{v}_f = R_x(\beta) \cdot R_z(\alpha) \cdot \vec{v}_0$$

$$\vec{v}_f = R_x(\beta) \cdot R_z(\alpha) \cdot \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \text{con} \quad R_x(\beta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{pmatrix} \quad \text{y} \quad R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Cálculo de la Trayectoria Final

Sustituyendo los valores ($\alpha = -20^\circ$, $\beta = 30^\circ$):

$$\vec{v}_f = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{pmatrix} \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -\sin \alpha \\ \cos \beta \cos \alpha \\ \sin \beta \cos \alpha \end{pmatrix} \approx \begin{pmatrix} 0,342 \\ 0,814 \\ 0,470 \end{pmatrix}$$

Nota: El tercer ángulo γ representaría la rotación propia sobre el eje de la trayectoria, la cual no altera el vector de dirección final. Por ejemplo, un avión volando de lado como cuando aterrizan en Bilbao.

6 Derivación numérica

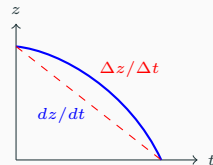
Derivadas en la Física de Videojuegos

- Una **derivada** nos dice cómo cambia una cantidad respecto a otra.
- Proporcionan una **tasa de cambio instantánea**.

Ejemplo: Caída Libre

Si z es la altitud y t el tiempo, la velocidad vertical es:

$$v_z = \frac{dz}{dt}$$



La derivada es la pendiente de la curva en cada punto.

Derivadas de Orden Superior

Es posible tomar la derivada de otra derivada.

- **Primer orden:** Define la velocidad (v) como el cambio de posición respecto al tiempo.
- **Segundo orden:** Define la aceleración (a) como el cambio de velocidad.

Relación Cinemática

La aceleración es la segunda derivada de la posición (z) con respecto al tiempo (t):

$$a_z = \frac{dv_z}{dt} = \frac{d^2z}{dt^2}$$

En un proyectil bajo gravedad constante, la aceleración es horizontal (constante), la velocidad aumenta linealmente y la altitud en función del tiempo es una función cuadrática (parábola).

Derivación Numérica: Diferencias Finitas

En computación, no podemos calcular límites infinitesimales ($dt \rightarrow 0$). En su lugar, usamos un **paso de tiempo discreto** Δt .

Aproximación por Diferencias

Si conocemos el valor de una función en instantes de tiempo separados por un intervalo Δt , la derivada se aproxima como:

$$f'(t) \approx \frac{f(t + \Delta t) - f(t)}{\Delta t}$$

- En videojuegos, Δt suele ser el tiempo entre frames (*delta time*).
- Cuanto más pequeño es Δt , más precisa es la aproximación.
- **Error de truncamiento:** Al ignorar los términos superiores de la serie de Taylor, cometemos un pequeño error numérico.

Esquemas de Diferencias Finitas

Existen tres esquemas básicos:

- **Hacia adelante:** Usa el punto siguiente.

$$f'(t) \approx \frac{f(t + \Delta t) - f(t)}{\Delta t}$$

- **Hacia atrás:** Usa el punto anterior (común en tiempo real).

$$f'(t) \approx \frac{f(t) - f(t - \Delta t)}{\Delta t}$$

- **Centrada:** Usa ambos extremos. Es más precisa.

$$f'(t) \approx \frac{f(t + \Delta t) - f(t - \Delta t)}{2\Delta t}$$

Aplicación en Física

Para calcular la velocidad actual v_i conociendo la posición actual x_i y la del frame anterior x_{i-1} :

$$v_i = \frac{x_i - x_{i-1}}{\Delta t}$$

Implementación: Cálculo de Velocidad

En cada ciclo de la simulación, podemos estimar la velocidad actual comparando la posición nueva con la del frame anterior:

Pseudo-código

```
// Inicialización
x_prev = x_inicial
dt = paso_de_tiempo // (p. ej. 1/60 seg)

BUCLE de simulación:
// 1. Obtener nueva posición del motor físico
x_actual = obtener_posicion()

// 2. Calcular velocidad (Diferencia hacia atrás)
v = (x_actual - x_prev) / dt

// 3. Actualizar estado para el siguiente ciclo
x_prev = x_actual
```

- Este método es fundamental cuando el motor nos da la posición pero no la velocidad instantánea.
- Por ejemplo, Chipmunk2D (pymunk) da la velocidad pero no la aceleración.

7 Integración numérica

El Problema de la Integración

En física y matemáticas, a menudo necesitamos calcular:

$$I = \int_a^b f(x) dx$$

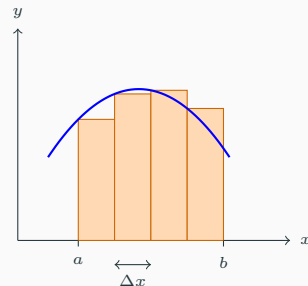
¿Por qué numérica?

- **Sin primitiva conocida:** Funciones como e^{-x^2} (distribución normal) no tienen una solución analítica simple.
- **Datos discretos:** A veces solo tenemos puntos medidos por un sensor (como la posición del péndulo de Pymunk).
- **Complejidad:** La función es tan compleja que es más rápido dejar que el ordenador aproxime el área.

¿En qué consiste el cálculo numérico?

La idea fundamental es la **discretización**:

1. Dividir el intervalo $[a, b]$ en n subintervalos pequeños de ancho Δx .
2. Aproximar el área de cada parte mediante una figura geométrica simple.
3. Sumar todas las áreas.

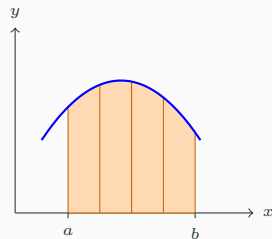
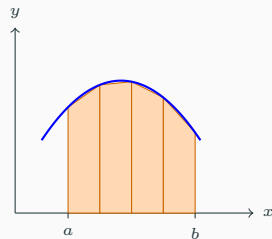


Suma de Riemann (4 intervalos)

- Este método se conoce como **regla del rectángulo**: asume que la función es constante en cada trozo. Error elevado.

Regla de los Trapecios y de Simpson

- Regla de los trapecios
- Aproxima el área bajo la curva usando **trapecios** en lugar de rectángulos.
 - Conecta los puntos $(x_i, f(x_i))$ con segmentos rectos.
 - Reduce el error respecto a los rectángulos.
- Regla de Simpson
 - Utiliza **parábolas** para aproximar la función en cada intervalo.
 - Requiere que los intervalos se agrupen de dos en dos.
 - Es mucho más precisa para funciones suaves.



Comparativa de Métodos

Para una división del intervalo en $\Delta x = (b - a)/n$:

Método	Geometría	Grado Polinomio	Error
Rectángulo	Rectángulo	0 (Constante)	$O(\Delta x)$
Trapezio	Trapezio	1 (Lineal)	$O(\Delta x^2)$
Simpson	Parábola	2 (Cuadrático)	$O(\Delta x^4)$

- El método de **Simpson** es sorprendentemente exacto: incluso integra perfectamente polinomios de grado 3, a pesar de usar parábolas.
- En sistemas físicos, elegir el método adecuado es clave para la **conservación de la energía**: un error acumulado alto puede hacer que los objetos "salgan volando." se detengan sin fricción.
- Cuanto mayor es el orden del error, más grande puede ser el Δt sin sacrificar la estabilidad de la simulación.

8 Resolución numérica de ecuaciones diferenciales

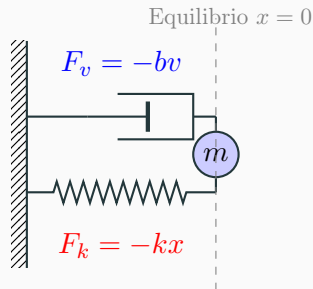
Ecuaciones Diferenciales Ordinarias (ODE)

Una **ODE** es una ecuación donde la incógnita es una función que depende de una sola variable (usualmente el tiempo t).

- En **Dinámica** suelen aparecer ecuaciones donde la incógnita es la posición en función del tiempo y parecen su primera y segunda derivada:
- La **posición** $x(t)$: Situación del objeto en el instante t .
- La **velocidad** $\frac{dx}{dt}$: Primera derivada de la posición.
- La **aceleración** $\frac{d^2x}{dt^2}$: Segunda derivada de la posición.

Resolver la ODE consiste en encontrar la función $x(t)$ que describe el movimiento del objeto.

Ejemplo. Oscilador Amortiguado



- **Fuerza Elástica (F_k):** Recuperadora, depende de la elongación x .
- **Fuerza Viscosa (F_v):** Disipativa, depende de la velocidad v .
- La suma de todas estas fuerzas determina la aceleración en cada momento:

$$ma = \sum F_i = -kx - bv$$

Solución Analítica: Oscilador Amortiguado

Ejemplo: Sistema Masa-Resorte-amortiguador

$$m \frac{d^2 x}{dt^2} - b \frac{dx}{dt} + kx = 0$$

Donde los parámetros definidos son:

- **Masa (m):** 0,2 kg.
- **Amortiguamiento (b):** 0,1 kg/s.
- **Constante (k):** 2,0 N/m.
- **Estado inicial:** Amplitud de 20 cm y reposo ($v_0 = 0$).

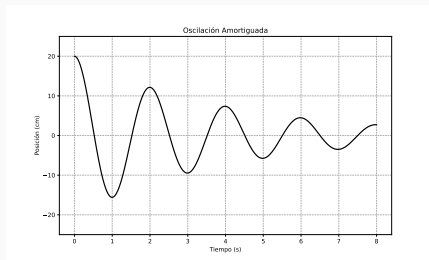


Figura 1: Comportamiento teórico del oscilador.

Esta solución representa el comportamiento físico ideal (exacto) donde la energía se disipa de forma continua.

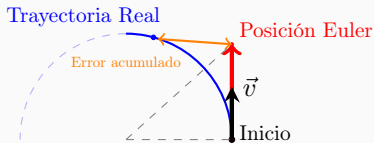
Resolución numérica de ODE: Newton (Euler) vs. Runge-Kutta

Como muchas ODEs no tienen solución analítica sencilla, usamos métodos numéricos para “dar paso” en el tiempo.

Método de Newton (Euler) Es el más simple (aproximación lineal):

$$x_{n+1} = x_n + v_n \Delta t$$

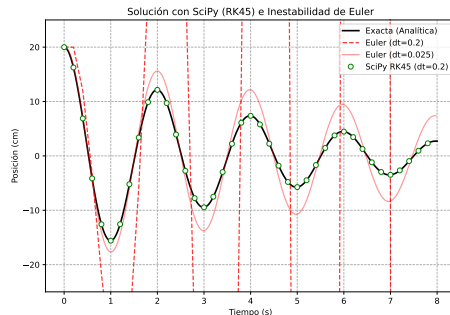
- **Pros:** Muy rápido de calcular.
- **Contras:** Inestable; acumula mucho error en trayectorias con mucha curvatura.



Método de Runge-Kutta (RK4) Evalúa la pendiente en varios puntos (4 pasos) para predecir el siguiente estado.

- **Pros:** Preciso y estable.
- **Contras:** Costoso computacionalmente
- **Runge-Kutta-Fehlberg (RK45)**
Compara soluciones de 4^º y 5^º orden para ajustar automáticamente el paso de tiempo (Δt). Precisión garantizada.

Resolución numérica de ODE: Oscilador Amortiguado



- **Datos:** Masa-resorte con amortiguamiento $b = 0,1 \text{ kg/s}$, $k = 2,0 \text{ N/m}$, $m = 0,2 \text{ kg}$.
- Con $\Delta t = 0,2 \text{ s}$, el método de **Euler** diverge de la trayectoria real.
- Con $\Delta t = 0,025 \text{ s}$ mejora un poco.
- El método **RK45** mantiene la precisión incluso con muestreos gruesos.

Implementación con scipy

```
1  from scipy.integrate import solve_ivp
2  # 1. Definir la funcion del sistema (dy/dt = f(t, y))
3  def sistema(t, y, m, k, b):
4      x, v = y
5      dxdt = v
6      dvdt = (-b*v - k*x) / m
7      return [dxdt, dvdt]
8  # 2. Configurar y lanzar el solver -> y0 es el estado inicial [x0, v0]
9  sol = solve_ivp(sistema,
10                 t_span=(0, 8),          # Intervalo de tiempo (inicio, fin)
11                 y0=[0.2, 0.0],          # Condiciones iniciales
12                 method='RK45',          # El metodo adaptativo
13                 args=(0.2, 2.0, 0.1),    # Parametros m, k, b
14                 rtol=1e-6)               # Tolerancia de error (paso adaptativo)
15  # 3. Acceder a los resultados
16  # sol.t contiene los tiempos que el algoritmo eligio
17  # sol.y[0] contiene las posiciones calculadas
```

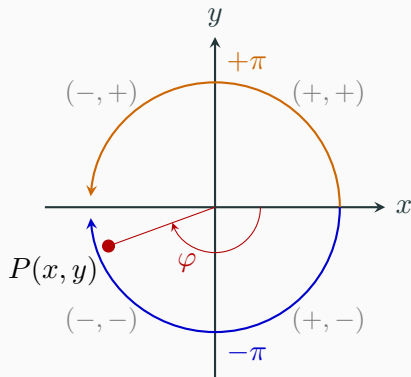
La función $\text{atan2}(y, x)$

La función **arcotangente de dos parámetros** es crucial en robótica y simulación porque:

- Evita la división por cero (cuando $x = 0$).
- Identifica el **cuadrante correcto** observando los signos de x e y .
- Devuelve un rango completo de $(-\pi, \pi]$ (o $[-180^\circ, 180^\circ]$).

Diferencia clave

- $\frac{-1}{-1}$ y $\frac{1}{1}$ dan 1. $\arctan(1)$ siempre devuelve 45° , pero $\text{atan2}(-1, -1)$ devuelve -135° .



Volver

Yaw (ψ) vs. Acimut (φ)

Aunque representan la misma rotación en el plano XY desde el eje $+X$, su interpretación numérica varía según el contexto:

Convención Física (φ)

- **Rango:** $[0, 2\pi)$.
- **Continuidad:** Salta de 2π a 0 al completar una vuelta.

Convención Robótica/Yaw (ψ)

- **Rango:** $(-\pi, \pi]$.
- **Continuidad:** Salta de π a $-\pi$ (el “corte” está atrás).

Implicaciones con `atan2(y, x)`

La función `atan2` devuelve por defecto el rango del **Yaw** $(-\pi, \pi]$.

- Si necesitas $\varphi \in [0, 2\pi)$ para una fórmula física: `if phi < 0: phi += 2*PI`
- **Control:** Es más fácil interpolar un Yaw (ej. de -170° a 170° son solo 20° de giro) que un ángulo acimutal que cruza el cero.

Volver